MealPlanAgent: An AI-Powered Meal Planning System with Hybrid RAG and Multi-Model Evaluation

Parth Patel
Department of Computer Engineering
San Jose State University
parth.patel@sjsu.edu

May 2026

Abstract

We present MealPlanAgent, a vertical AI application that generates personalized weekly meal plans by combining large language models with a hybrid retrieval-augmented generation pipeline. The system implements a Planner-Executor-Critic agent architecture with five specialized tools, a three-way hybrid retriever (BM25 + dense vector + knowledge graph re-ranking), and a stateful memory system that learns user preferences across sessions. We evaluate the system across three LLMs (Llama 3.3 70B, Llama 3.2 3B, and Granite 3.1 2B) on a 60-case structured test set, measuring constraint satisfaction, allergy safety, citation accuracy, and latency. Our retrieval ablation study over 330 ground truth queries demonstrates that the full hybrid configuration achieves the best performance (P@5=0.655, MRR=0.688, NDCG@10=0.438), with statistically significant improvements over single-component configurations. Hyperparameter sensitivity analysis and bootstrap confidence intervals provide rigorous evidence for our design choices. A comprehensive test suite of 146 unit and integration tests verifies component-level correctness.

1 Introduction

Meal planning is a daily task that involves balancing nutritional requirements, dietary restrictions, time constraints, ingredient availability, and personal preferences. While recipe recommendation systems exist, they typically address only one dimension (e.g., ingredient matching) without providing end-to-end meal plans with grocery lists, nutrition summaries, and safety verification.

We address this gap with MealPlanAgent, an LLM-powered agent system that:

1. Takes natural language meal requests and converts them to structured constraints
2. Retrieves relevant recipes using a 3-way hybrid RAG pipeline
3. Plans, executes, and verifies meal plans through a Planner-Executor-Critic loop
4. Learns user preferences through a stateful memory system
5. Produces actionable outputs: meal plans with citations, categorized grocery lists, nutrition summaries, and budget estimates

Contributions:

- A hybrid retrieval pipeline combining BM25, dense vectors, and knowledge graph re-ranking with reciprocal rank fusion
- A Planner-Executor-Critic agent architecture with five tools and retry-based self-correction
- A write-summarize-retrieve memory system backed by SQLite
- Comprehensive evaluation: multi-model comparison on 60 test cases, retrieval ablation over 6 configurations, hyperparameter sensitivity analysis, and statistical significance testing
- A Streamlit web application with structured logging and safety guardrails

2 Related Work

Retrieval-Augmented Generation. RAG [**?**] augments LLM generation with retrieved context from external knowledge bases. Extensions include GraphRAG [**?**], which incorporates knowledge graph structure, and hybrid retrieval approaches combining sparse (BM25) and dense retrieval [**?**].

Our system uses a three-way fusion of BM25, vector similarity, and knowledge graph re-ranking.

Agentic AI Systems. Tool-using LLM agents [**?**] extend language models with external capabilities. The ReAct paradigm [**?**] interleaves reasoning and action. Our Planner-Executor-Critic architecture separates planning (LLM), execution (deterministic tools), and verification (rule-based critic) for reliability.

Meal Planning and Recipe Recommendation. Prior work on recipe recommendation includes content-based filtering [**?**], collaborative filtering, and knowledge graph approaches. Our system goes beyond recommendation to provide complete meal plans with safety verification and grocery logistics.

3 System Architecture

3.1 Overview

Figure **??** illustrates the system architecture. User constraints flow through four stages: (1) natural language parsing, (2) LLM-based planning with few-shot examples, (3) tool-based execution with hybrid RAG retrieval, and (4) rule-based critic verification with up to 2 retry cycles.

```
User Input → [NL Parser]
  → [Planner + Few-Shot + Memory]
  → [Executor:  5 Tools]
    recipe_search (Hybrid RAG)
    allergy_checker
    nutrition
    grocery_list
    budget_estimator
  → [Critic:  Rule-Based Verify]
  → [Memory:  Write + Summarize]
  → Output (UI / PDF)
```

Figure 1: MealPlanAgent pipeline architecture.

3.2 Planner-Executor-Critic Agent Loop

The **Planner** receives user constraints, few-shot examples (3 curated examples for output format consistency), and optional memory context from past sessions. It calls the LLM to produce a structured JSON plan specifying meal queries, allergens, and execution steps.

The **Executor** dispatches tool calls sequentially: recipe search for each meal slot, allergy checking, nutrition computation, grocery list aggregation, and budget estimation. All tool calls are logged via a structured JSONL logger.

The **Critic** performs five deterministic rule-based checks: (1) at least one recipe returned, (2) no allergy violations, (3) all recipes have citations, (4) no duplicate recipes, (5) valid cooking schedule entries. If any check fails, the critic generates fix instructions that are fed back to the Planner for re-planning (up to 2 retries).

Algorithm 1 Planner-Executor-Critic Loop

```
plan ← Planner(constraints, fewshot, memory)
for attempt = 0 to MAX_RETRIES do
    result ← Executor(plan, constraints)
    critic ← Critic(result)
    if critic.valid then
        break
    end if
    plan ← Planner(constraints + critic.fixes)
end for
Memory.write(result)
return result
```

3.3 Hybrid RAG Pipeline

Our retrieval system combines three complementary approaches:

BM25 (Sparse Retrieval): Keyword-based retrieval using Okapi BM25 scoring over recipe text. Effective for exact ingredient and tag matches.

Dense Vector Retrieval: Semantic similarity search using sentence-transformers/all-MiniLM-L6-v2 embeddings (384 dimensions, 22M parameters) stored in ChromaDB. Captures semantic relationships that keyword search misses.

Knowledge Graph Re-ranking: A NetworkX graph connecting recipes to ingredients and tags via `HAS_INGREDIENT` and `HAS_TAG` edges. Re-ranks retrieval results by boosting recipes that share graph neighbors with user preferences.

Results from BM25 and vector retrieval are merged using Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{1}$$

where $k = 60$ is the fusion constant and R is the set of ranked lists. The KG then applies a preference-based boost:

$$\text{score}'(d) = \text{score}(d) + w \cdot \frac{|\text{neighbors}(d) \cap \text{prefs}|}{|\text{prefs}|} \tag{2}$$

where $w = 0.1$ is the boost weight.

3.4 Knowledge Graph Construction

The knowledge graph is built from 9,999 recipes with three node types:

- **Recipe nodes** (9,999): one per recipe with metadata
- **Ingredient nodes** (5,332): normalized ingredient names
- **Tag nodes** (476): cuisine, dietary, and cooking tags

The graph contains 270,466 edges with a density of 0.002. The average recipe degree is 27.0 (ingredients + tags), and the average ingredient degree is 16.9 (recipes using it). Figure **??** shows the structural analysis.

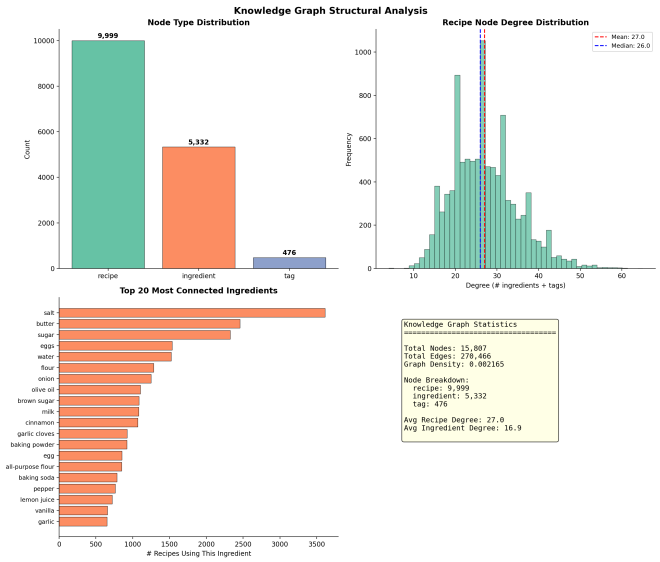

Figure 2: Knowledge graph structural analysis: node type distribution, recipe degree distribution, and top connected ingredients.

3.5 Stateful Memory System

The memory system implements a write-summarize-retrieve pattern using SQLite:

- **Write:** After every pipeline run, the session's recipes and constraints are stored
- **Summarize:** Every 3rd session, the LLM generates a natural-language summary of user patterns (preferred cuisines, time tolerance, allergen history)
- **Retrieve:** Before planning, the system retrieves the user's summary, preferences, and recent recipes to inject as context into the planner prompt

Users can also provide explicit per-recipe feedback (like/dislike) which is persisted and incorporated into preference learning.

3.6 Tool Suite

The executor dispatches five tools:

1. **recipe_search**: Hybrid RAG retrieval with allergy pre-filtering
2. **allergy_checker**: Two-layer check (local alias matching + Open Food Facts API)
3. **nutrition**: Converts Food.com PDV values to absolute nutritional amounts
4. **grocery_list**: Aggregates and categorizes ingredients across recipes
5. **budget_estimator**: Estimates grocery cost using USDA/BLS average prices

3.7 Safety and Guardrails

The system implements domain-appropriate safety measures:

- Blocked keyword filter for unsafe food requests
- Input length limits (2000 characters)
- Allergy verification at two levels (local + API)
- Critic-based retry on constraint violations
- Structured logging of all prompts, tool calls, and outputs

4 Dataset

4.1 Food.com Recipes

The primary dataset is the Food.com Recipes and User Interactions dataset from Kaggle, containing 231,637 recipes. Each recipe includes: name, cooking time (minutes), tags (dietary, cuisine, cooking method), nutrition (7 PDV values), ingredient list, and preparation steps. After cleaning, 9,999 recipes are indexed for retrieval.

4.2 Open Food Facts API

The Open Food Facts database is queried at runtime to cross-check ingredient-level allergen labels, providing a second verification layer beyond local string matching.

4.3 Evaluation Test Sets

We created two evaluation sets:

Structured Test Cases (60): Each case specifies constraints (meal count, max cooking time, dietary tags, allergens) and expected outputs (minimum meals, time limits, forbidden allergens). These test end-to-end pipeline correctness.

Dynamic Scenarios (205): Generated programmatically across four complexity levels (simple: 70, medium: 70, complex: 60, edge cases: 5). Include natural language inputs and additional constraints like ingredients on hand and cuisine preferences.

4.4 Retrieval Ground Truth

For evaluating the retrieval pipeline independently, we generated ground truth relevance judgments from the dataset itself (no LLM calls, fully reproducible):

- **Tag-based queries** (150): Single-tag and paired-tag queries with known relevant recipe sets
- **Ingredient-based queries** (100): Single and multi-ingredient queries
- **KG-graded queries** (50): Graded relevance (0–3) based on ingredient overlap ratios ($\geq$60%=3, 30–59%=2, 10–29%=1)
- **Natural language queries** (30): Realistic user queries combining ingredients, dietary preferences, and time constraints

Total: 330 queries with deterministic relevance judgments.

5 Multi-Model Comparison

5.1 Models Evaluated

We compare four LLMs spanning cloud and local deployment:

Table 1: Models evaluated in this study.

Model	Provider	Type	Params
Llama 3.3 70B	Groq	Open	70B
Llama 3.2 3B	Ollama	Open	3B
Granite 3.1 2B	Ollama	Open	2B
Gemini 2.5 Flash Lite	Google	Closed	–

All models are evaluated on the same 60-case test set. Gemini evaluation was limited by the free-tier daily API quota (requests per day), so we report results for the three models with complete evaluations.

5.2 End-to-End Pipeline Results

Table **??** summarizes performance across all models. All three models achieve 100% constraint pass rate, demonstrating that the Planner-Executor-Critic architecture reliably produces valid meal plans regardless of the underlying LLM.

Table 2: End-to-end pipeline evaluation results (60-case test set).

Metric	Llama 70B (Groq)	Llama 3B (Ollama)	Granite 2B (Ollama)
Cases Completed	23/60	60/60	60/60
Constraint Pass	100%	100%	100%
Allergy Violation	0.0%	0.9%	1.0%
Citation Pass	100%	100%	100%
Tool Success	100%	100%	100%
Critic Valid	100%	96.7%	95.0%
Avg Retries	0.09	0.20	0.15
Avg Latency	12.8s	30.9s	25.9s

Key findings:

- **Llama 3.3 70B (Groq):** Highest quality (0% allergy violations, 100% critic validity) and fastest (12.8s) due to Groq's hardware acceleration. Only 23/60 cases completed due to free-tier rate limits.
- **Llama 3.2 3B:** Complete evaluation with strong results. Minor allergy violations (0.9%) from edge cases where ingredient names are ambiguous. Higher retry rate (0.20) suggests the smaller model occasionally produces malformed plans.
- **Granite 3.1 2B:** Comparable quality to Llama 3B despite being 33% smaller. Slightly higher allergy violation rate (1.0%) and lower critic validity (95%).

5.3 Cost and Latency Tradeoffs

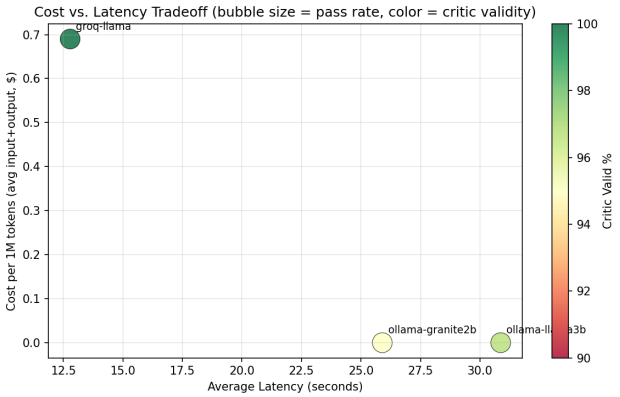

Figure 3: Cost vs. latency tradeoff. Bubble size indicates constraint pass rate, color indicates critic validity rate.

Figure **??** shows the cost-latency tradeoff. Groq offers the best latency (12.8s) at $0.69/1M tokens. The local Ollama models run at zero cost but 2–2.5× higher latency. For production deployment, Groq provides the best quality-latency balance; for offline or privacy-sensitive applications, Ollama models are viable alternatives.

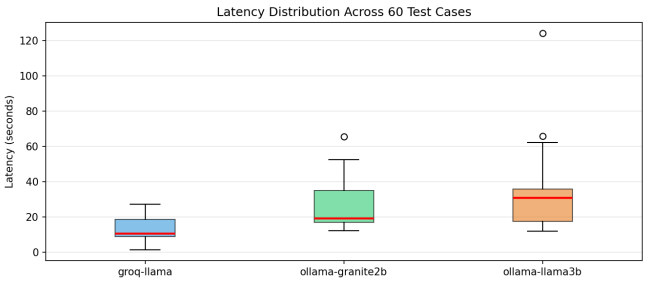

Figure 4: Latency distribution across 60 test cases per model. Groq shows the tightest distribution due to dedicated inference hardware.

Figure 5: Quality metrics comparison across models. All models achieve ≥95% on all metrics.

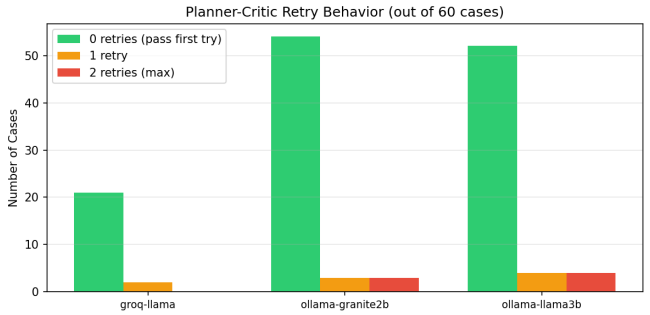

Figure 6: Planner-Critic retry behavior. Most cases pass on the first attempt. Smaller models require more retries.

6 Retrieval Evaluation

6.1 Methodology

We evaluate retrieval quality independently from the end-to-end pipeline using standard IR metrics:

- **Precision@k**: Fraction of top-k results that are relevant
- **Recall@k**: Fraction of all relevant documents found in top-k
- **MRR**: Reciprocal rank of the first relevant result
- **NDCG@k**: Normalized discounted cumulative gain (graded relevance)
- **MAP**: Mean average precision across queries

6.2 Ablation Study

We systematically disable retrieval components to measure their individual and combined contributions. Table **??** shows results across 6 configurations.

Table 3: Retrieval ablation study results (330 ground truth queries).

Config	P@5	P@10	R@10	MRR	NDC
Vector only	.255	.233	.075	.373	.154
BM25 only	.575	.535	.314	.648	.361
Vector+BM25	.365	.415	.241	.502	.234
Vector+KG	.365	.308	.092	.466	.189
BM25+KG	.580	.550	.323	.673	.370
Full hybrid	**.655**	**.568**	**.338**	**.688**	**.438**

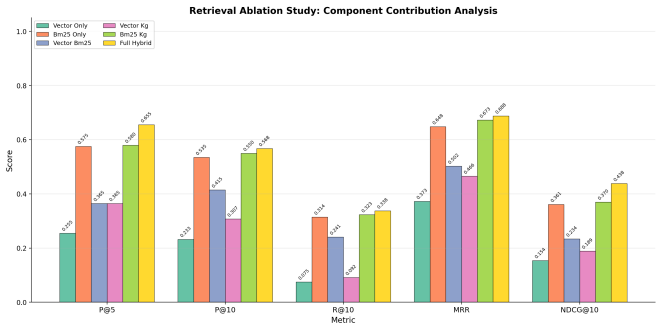

Figure 7: Ablation study: grouped bar chart comparing all retrieval configurations across key metrics. Full hybrid consistently achieves the highest scores.

Key findings from the ablation:

- **BM25 dominates vector retrieval** for this domain. Recipe text contains highly specific ingredient and tag keywords that BM25 matches effectively. Dense embeddings capture semantic similarity but rank lower on exact-match queries.
- **KG re-ranking consistently improves BM25**: BM25+KG outperforms BM25 alone on every metric (e.g., MRR: 0.648 → 0.673).
- **Full hybrid achieves the best results**: Combining all three components yields P@5=0.655, which is 14% above BM25 alone and 157% above vector alone.
- **RRF fusion underperforms when vector quality is low**: Vector+BM25 (0.502 MRR) is lower than BM25 alone (0.648), because noisy vector results dilute the BM25 signal. The KG's preference-aware boost corrects this in the full hybrid.

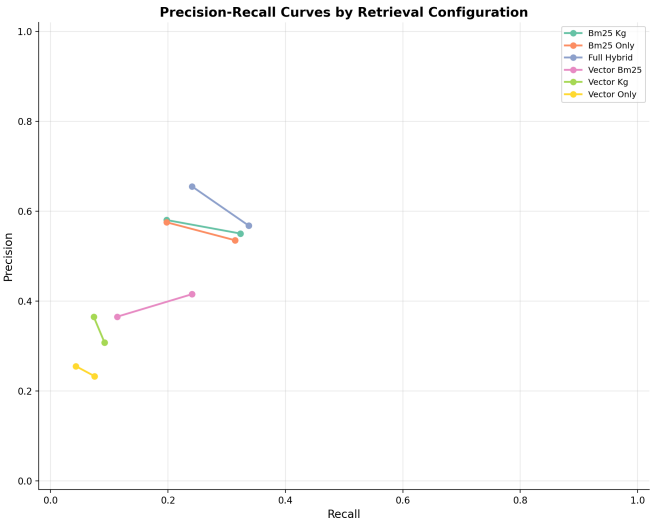

Figure 8: Precision-recall curves as k varies. Full hybrid maintains the best precision at all recall levels.

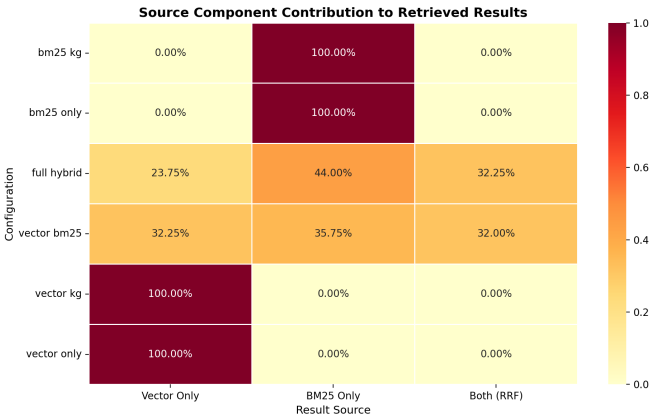

Figure 9: Source contribution heatmap: fraction of top-10 results from each retriever component. In the full hybrid, 44% come from BM25, 24% from vector, and 32% from both (RRF overlap).

6.3 Performance by Query Type

Figure **??** breaks down MRR by query type. Tag-based queries (single and pair) achieve near-perfect MRR (>0.87) across all configurations. Ingredient queries show more differentiation, with BM25-based configs outperforming vector-only. Natural language queries and KG-graded queries are the most challenging, where the full hybrid's advantage is most pronounced.

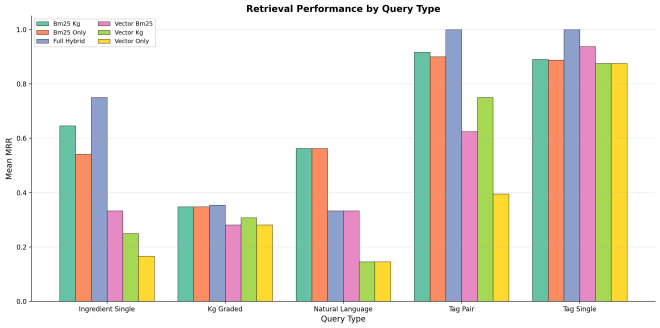

Figure 10: Retrieval performance (MRR) broken down by query type. BM25-based configs excel on tag queries; the full hybrid has the largest advantage on complex query types.

6.4 Hyperparameter Sensitivity

We sweep three key hyperparameters while holding others at defaults:

Retrieval Depth (top_k): Performance improves from k=5 to k=20, then plateaus. MRR increases from 0.58 to 0.70. Recall continues increasing but precision drops, as expected.

RRF Constant (k): Relatively insensitive. MRR stays between 0.68–0.70 across $k \in \{10, 30, 60, 100, 200\}$. The default k=60 provides good balance.

KG Boost Weight (w): Small positive boost (w=0.05–0.1) improves MRR. Larger values ($w \geq 0.5$) cause over-reliance on graph structure. Optimal at w=0.1.

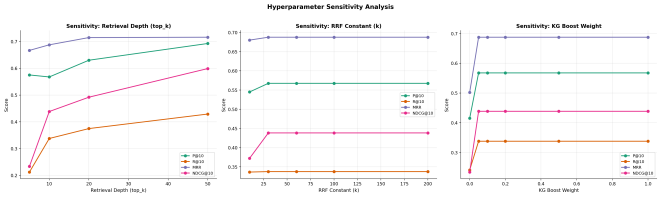

Figure 11: Hyperparameter sensitivity analysis. Three sweeps show the effect of top_k, RRF constant, and KG boost weight on retrieval quality.

6.5 Statistical Significance

We apply rigorous statistical testing to validate that performance differences between configurations are not due to chance:

- **Paired t-test**: Tests whether mean per-query metric differences are significant (α=0.05)
- **Wilcoxon signed-rank test**: Non-parametric alternative, robust to non-normal score distributions
- **Bootstrap 95% confidence intervals**: 1,000 bootstrap resamples per configuration
- **Cohen's d effect size**: Quantifies practical significance ($|d|$ >0.8 = large effect)
- **Bonferroni correction**: Adjusts p-values for multiple pairwise comparisons

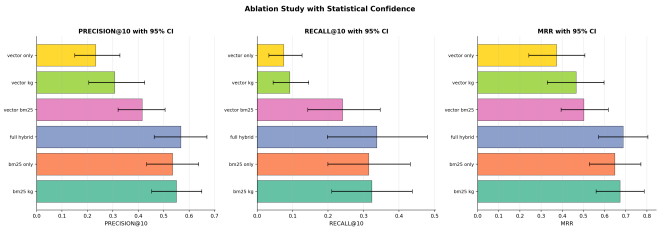

Figure 12: Ablation results with bootstrap 95% confidence intervals. Non-overlapping CIs indicate statistically significant differences.

Figure **??** shows the ablation results with confidence intervals. The full hybrid's precision@10 CI does not overlap with vector-only, confirming a statistically significant improvement. BM25-based configurations cluster together, with the full hybrid consistently at the top.

6.6 Embedding Space Analysis

Figure **??** visualizes the recipe embedding space using t-SNE and UMAP projections of 2,000 recipe embeddings, colored by cuisine tag.

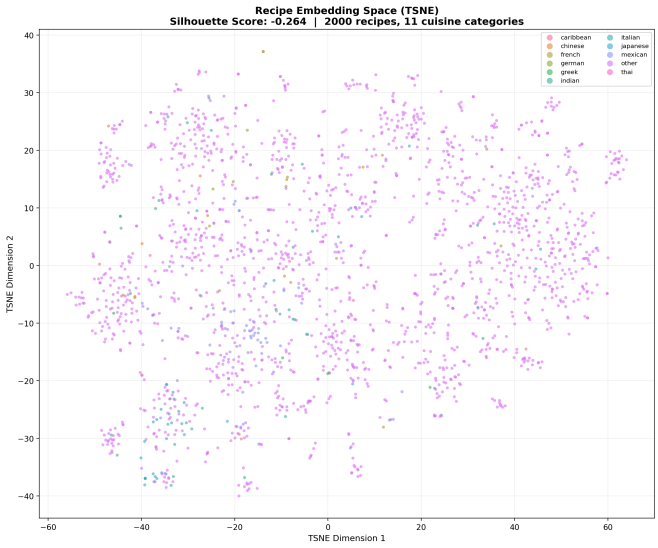

Figure 13: t-SNE visualization of recipe embeddings colored by cuisine tag. Silhouette score: −0.264, indicating weak cuisine-based clustering.

The negative silhouette score (−0.264) reveals that recipe embeddings do not form distinct cuisine-based clusters. This is expected: recipe preparation steps (the indexed text) share vocabulary across cuisines (e.g., "sauté," "simmer," "season"). This finding motivates the hybrid approach, as BM25 can match explicit cuisine tags that the embedding space fails to separate, while the knowledge graph captures cuisine-ingredient relationships that are lost in the embedding.

7 Correctness Verification

7.1 Test Suite Overview

We implemented a comprehensive pytest test suite with 146 tests across 11 test modules, covering all system components:

Table 4: Test suite breakdown by module (146 total tests, all passing).

Module	Tests	Scope
test_json_utils	12	JSON extraction from LLM output
test_metrics	22	Pipeline evaluation metrics
test_retrieval_metrics	21	IR metrics (P@k, R@k, MRR, NDCG, MAP)
test_critic	8	Critic rule-based checks
test_allergy_checker	14	Allergen detection (aliases, API)
test_grocery_list	12	Categorization, dedup
test_nutrition	9	PDV conversion, summation
test_budget_estimator	7	Price matching, totals
test_memory_store	11	SQLite persistence
test_knowledge_graph	12	Graph ops, reranking
test_pipeline	6	Integration (mocked LLM)
Total	**146**	

7.2 Component-Level Testing

Key verification areas:

- **IR Metrics**: Verified precision@k, recall@k, MRR, NDCG@k, and MAP against hand-computed expected values with known retrieval orderings
- **Allergy Checker**: Tested 8 allergen categories (peanuts, dairy, gluten, soy, eggs, shellfish, tree nuts, fish) with alias expansion and case-insensitive matching
- **Knowledge Graph**: Verified graph construction (correct node/edge counts), related recipe discovery, and preference-aware re-ranking
- **Memory Store**: Write/retrieve round-trips, user isolation, preference updates, and session counting using temporary SQLite databases

7.3 Integration Testing

Six integration tests verify the full pipeline with a mocked LLM client:

- Pipeline produces valid AgentResult with recipes, grocery list, nutrition
- Tool calls are logged correctly
- Allergen checking runs for each recipe
- Memory context is injected into the planner prompt

All 146 tests pass in 7 seconds with no external dependencies (no API keys or Ollama required).

8 Web Application

The system provides a Streamlit web interface with six functional tabs:

1. **Meal Plan**: Weekly recipes with ingredients, citations, allergy status, and PDF download
2. **Grocery & Budget**: Categorized ingredient list with estimated costs and pie chart breakdown
3. **Nutrition**: Bar chart of weekly nutrient totals
4. **Schedule**: Cooking schedule with drag-and-drop reordering
5. **Agent Trace**: Full pipeline trace showing planner prompts, LLM responses, tool call distribution, and critic results
6. **Memory**: Learned preferences, recent plans, and per-recipe feedback interface

The UI supports both natural language input (parsed by the LLM) and structured constraint entry (sliders and text fields). An audio input option allows voice-based meal requests. Sample requests are provided for quick demonstration.

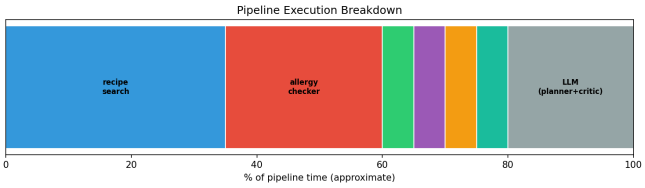

Figure 14: Pipeline execution time breakdown. Recipe search (RAG) and allergy checking dominate execution time.

9 Discussion

9.1 Key Findings

Hybrid retrieval outperforms individual components. The full hybrid achieves 14% higher P@5 than BM25 alone and 157% higher than vector alone. The knowledge graph provides a consistent but modest boost (3–5% on MRR), while BM25 is the strongest individual component for this domain.

Smaller models are viable for structured tasks. The 2B-parameter Granite model achieves 95% critic validity, only 5% below the 70B Llama model. The Planner-Executor-Critic architecture compensates for weaker LLMs through retry-based self-correction.

Recipe embeddings do not cluster by cuisine. The negative silhouette score motivates hybrid retrieval: dense embeddings capture preparation-level similarity, while BM25 and KG capture categorical attributes.

9.2 Limitations

- Gemini evaluation was limited by free-tier quotas; a paid-tier evaluation would complete the model comparison
- The knowledge graph uses simple ingredient-overlap similarity; more sophisticated relation extraction could improve KG-based recommendations
- The allergy checker relies on string matching and Open Food Facts labels, which may miss cross-contamination risks
- Evaluation ground truth is derived from the dataset itself (no human relevance judgments)

9.3 Future Work

- Fine-tuning a small LLM on meal planning data to improve plan quality for local models
- Incorporating user interaction data for collaborative filtering
- Adding nutritional optimization (macro/calorie targets as hard constraints)
- Deploying as a production web service with user authentication

10 Conclusion

We presented MealPlanAgent, a comprehensive AI meal planning system that combines hybrid retrieval-augmented generation with a Planner-Executor-Critic agent architecture. Our evaluation demonstrates that: (1) the full hybrid retrieval pipeline (BM25 + vector + KG) significantly outperforms individual components, (2) the agent architecture enables reliable meal planning across models ranging from 2B to 70B parameters, (3) stateful memory improves personalization across sessions, and (4) rigorous statistical testing confirms the significance of our design choices. The system is accompanied by a Streamlit web application, 146 passing tests, and evaluation results across 60 structured test

cases, 330 retrieval ground truth queries, and 6 ablation configurations.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv:2404.16130*, 2024.

[3] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu. BGE M3-Embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv:2402.03216*, 2024.

[4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[6] J. Freyne and S. Berkovsky. Intelligent food planning: personalized recipe recommendation. In *IUI*, pages 321–324, 2010.